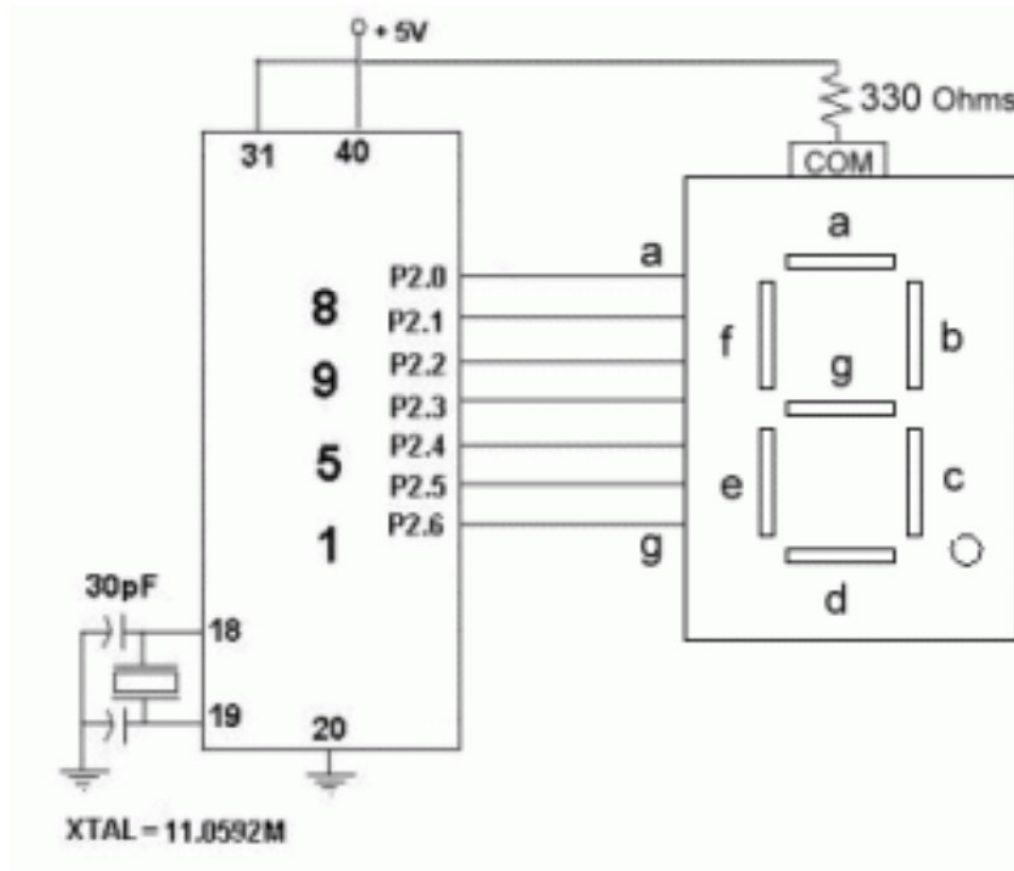


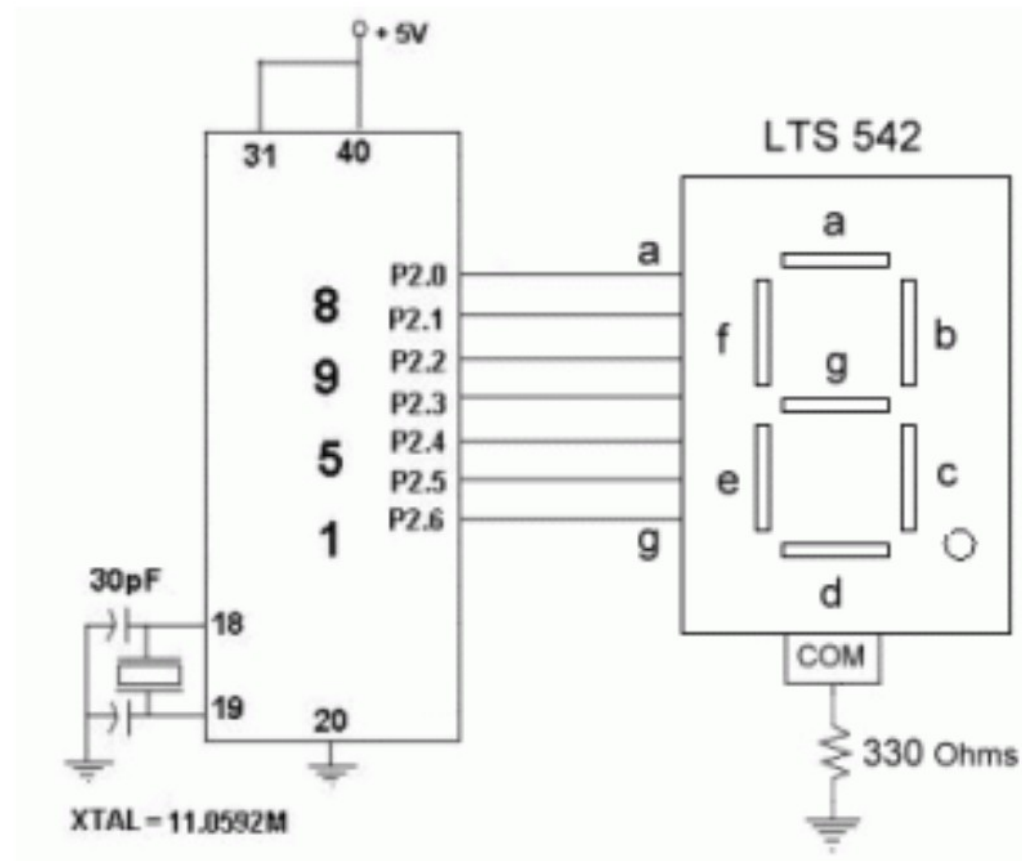
This week

- Quiz
- Multi-digit 7-Segment display
- Hardware Timers/Counters

Common Anode



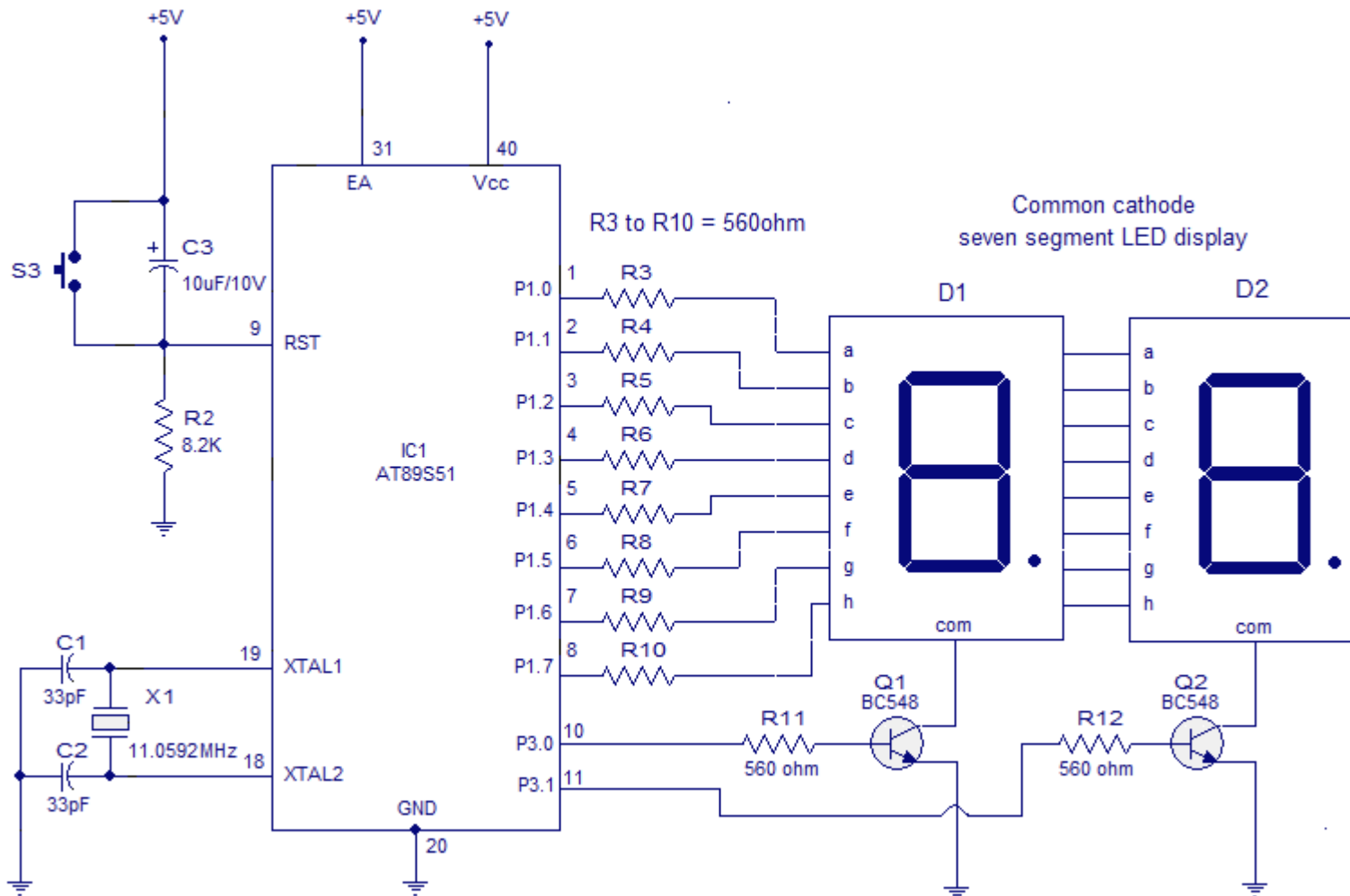
Common Cathode



Simple example

- MOV P1,#0B0H ; display 3
- MOV P1,#99H ; display 4

2-digit 7-Segment Display



Multiplexing 7 segment displays to 8051

www.circuitstoday.com

Test

- Blink LEDs with some delay
- Count objects passing on a belt. Each passing object generates a pulse on Pin 3.5.

Timers/Counter

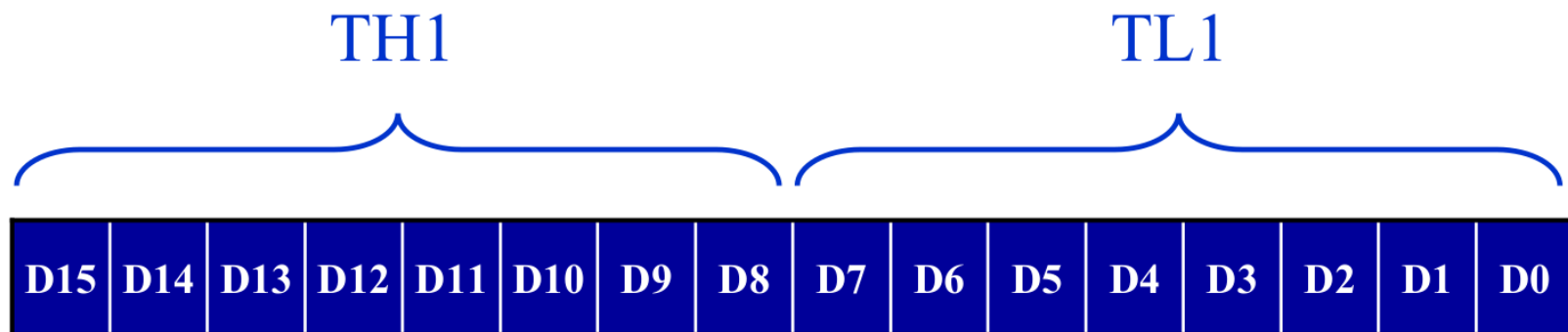
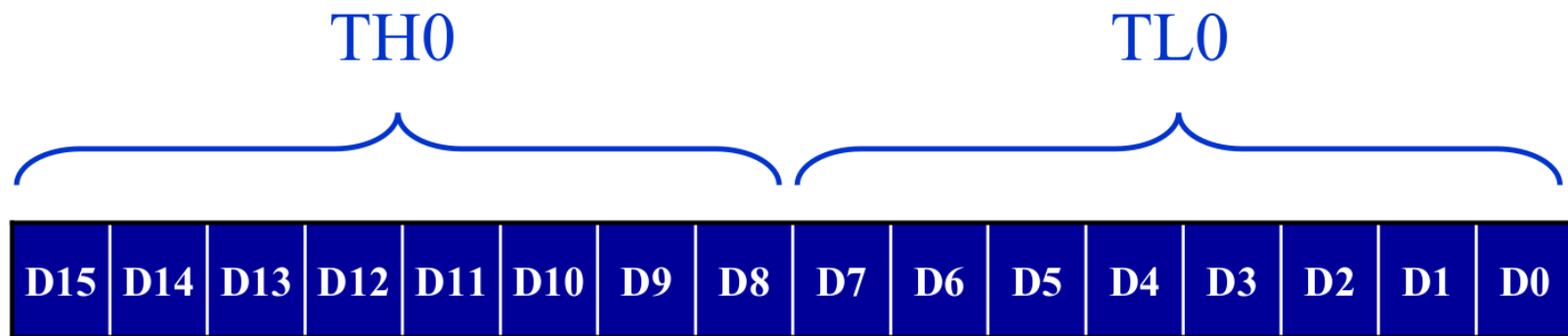
- 2 timers/counters (Timer 0, Timer 1)
- 16-bit wide (Two separate 8-bit registers for low byte and high byte)
- Uses
 - Calculate amount of time between events
 - Counting events
 - Generate baud rate for serial port

Important Registers

- TMOD in SFR is used to set Timer Mode
- TCON in SFR is used to control timer
- THx and TLx hold the count

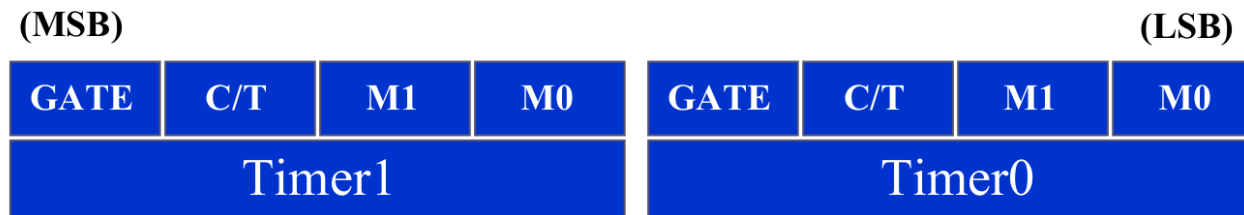
THx/TLx

- The low byte register is called TL0/TL1 and
- The high byte register is called TH0/TH1
- Accessed like any other register
 - `MOV TL0, #4FH`
 - `MOV R5, TH0`



TMOD

- ❑ Both timers 0 and 1 use the same register, called TMOD (timer mode), to set the various timer operation modes
- ❑ TMOD is a 8-bit register
 - The lower 4 bits are for Timer 0
 - The upper 4 bits are for Timer 1
 - In each case,
 - The lower 2 bits are used to set the timer mode
 - The upper 2 bits to specify the operation



TMOD

TMOD : Timer/Counter Mode Control Register (Not Bit Addressable)

GATE	C/ $\overline{T}$	M1	M0	GATE	C/ $\overline{T}$	M1	M0
TIMER 1				TIMER 0			

GATE	When TRx (in TCON) is set and GATE = 1, TIMER/COUNTERx will run only while INTx pin is high (hardware control). When GATE = 0, TIMER/COUNTERx will run only while TRx = 1 (software control).
C/ $\overline{T}$	Timer or Counter selector. Cleared for Timer operation (input from internal system clock). Set for Counter operation (input from Tx input pin).
M1	Mode selector bit (NOTE 1).
M0	Mode selector bit (NOTE 1).

Note 1 :

M1	M0	OPERATING MODE	
0	0	0	13-bit Timer
0	1	1	16-bit Timer/Counter
1	0	2	8-bit Auto-Reload Timer/Counter
1	1	3	(Timer 0) TL0 is an 8-bit Timer/Counter controlled by the standard Timer 0 control bits, TH0 is an 8-bit Timer and is controlled by Timer 1 control bits.
1	1	3	(Timer 1) Timer/Counter 1 stopped.

Test

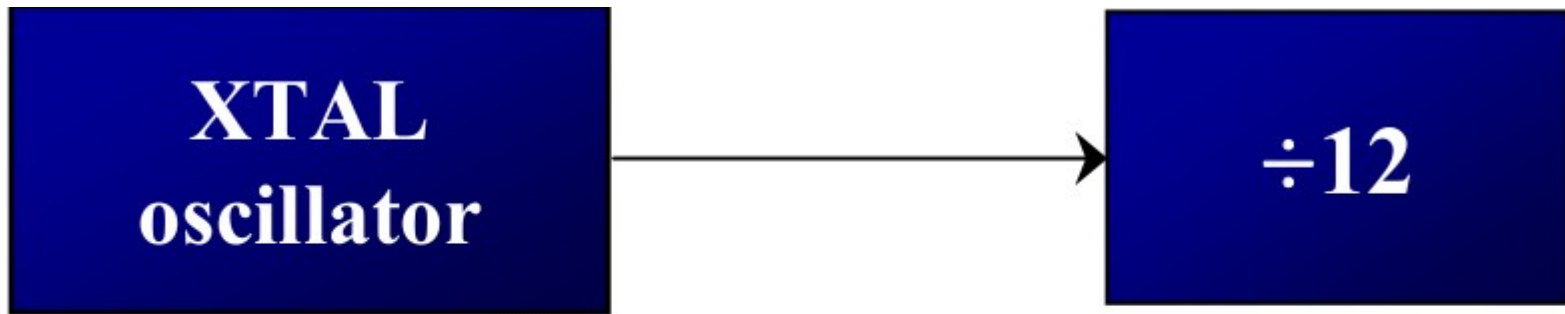
- Indicate which mode and which timer are selected for each of the following.

(a) MOV TMOD, #01H

(b) MOV TMOD, #20H

(c) MOV TMOD, #12H

Timer's Clock Frequency



$$1/12 \times 11.0529 \text{ MHz} = 921.6 \text{ MHz};$$
$$T = 1/921.6 \text{ kHz} = 1.085 \text{ us}$$

Timer control

- Start with SW (GATE=0)
 - SETB TR0
 - CLR TR0
- Start with HW (GATE=1)

Practice

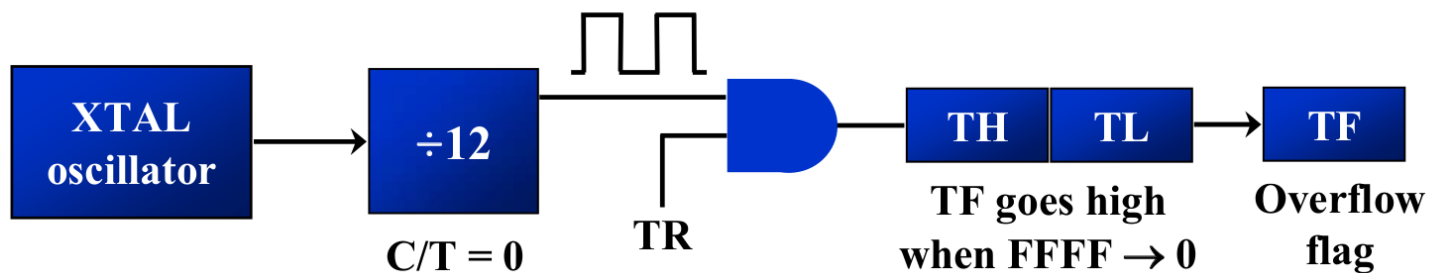
- Find the value for TMOD if we want to program timer 0 in mode 2, use 8051 XTAL for the clock source, and use instructions to start and stop the timer.

Solution

- MOV TMOD, 02H

Mode 1

- ❑ The following are the characteristics and operations of mode1:
 1. It is a 16-bit timer; therefore, it allows value of 0000 to FFFFH to be loaded into the timer's register TH and TL
 2. After TH and TL are loaded with a 16-bit initial value, the timer must be started
 - This is done by `SETB TR0` for timer 0 and `SETB TR1` for timer 1
 3. After the timer is started, it starts to count up
 - It counts up until it reaches its limit of FFFFH



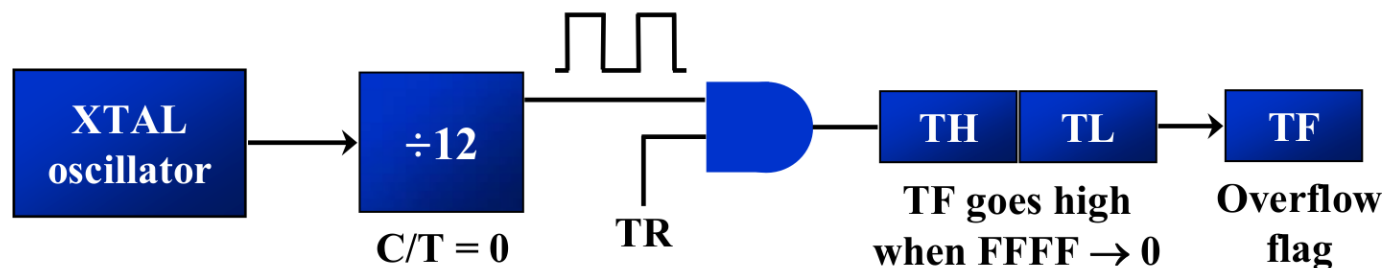
Mode 1

3. (cont')

- When it rolls over from FFFFH to 0000, it sets high a flag bit called TF (timer flag)
 - Each timer has its own timer flag: TF0 for timer 0, and TF1 for timer 1
 - This timer flag can be monitored
- When this timer flag is raised, one option would be to stop the timer with the instructions `CLR TR0` or `CLR TR1`, for timer 0 and timer 1, respectively

4. After the timer reaches its limit and rolls over, in order to repeat the process

- TH and TL must be reloaded with the original value, and
- TF must be reloaded to 0



Time Delay Generation

1. Load the TMOD value register indicating which timer (timer 0 or timer 1) is to be used and which timer mode (0 or 1) is selected
2. Load registers TL and TH with initial count value
3. Start the timer
4. Keep monitoring the timer flag (TF) with the `JNB TFx, target` instruction to see if it is raised
 - Get out of the loop when TF becomes high
5. Stop the timer
6. Clear the TF flag for the next round
7. Go back to Step 2 to load TH and TL again

Example

In the following program, we create a square wave of 50% duty cycle (with equal portions high and low) on the P1.5 bit. Timer 0 is used to generate the time delay. Analyze the program

```
                MOV    TMOD, #01      ;Timer 0, mode 1 (16-bit mode)
HERE:           MOV    TL0, #0F2H     ;TL0=F2H, the low byte
                MOV    TH0, #0FFH     ;TH0=FFH, the high byte
                CPL     P1.5           ;toggle P1.5
                ACALL   DELAY
                SJMP    HERE
```

In the above program notice the following step.

1. TMOD is loaded.
2. FFF2H is loaded into TH0-TL0.
3. P1.5 is toggled for the high and low portions of the pulse.

Example

DELAY:

```
        SETB  TR0          ;start the timer 0
AGAIN:  JNB   TF0, AGAIN    ;monitor timer flag 0
                        ;until it rolls over
        CLR   TR0          ;stop timer 0
        CLR   TF0          ;clear timer 0 flag
        RET
```

4. The DELAY subroutine using the timer is called.
5. In the DELAY subroutine, timer 0 is started by the SETB TR0 instruction.
6. Timer 0 counts up with the passing of each clock, which is provided by the crystal oscillator. As the timer counts up, it goes through the states of FFF3, FFF4, FFF5, FFF6, FFF7, FFF8, FFF9, FFFA, FFFB, and so on until it reaches FFFFH. One more clock rolls it to 0, raising the timer flag (TF0=1). At that point, the JNB instruction falls through.



7. Timer 0 is stopped by the instruction CLR TR0. The DELAY subroutine ends, and the process is repeated.

Notice that to repeat the process, we must reload the TL and TH registers, and start the process is repeated ...